1. Analysis

1.1 Brief in summary

Pennine Pizza Works needs staff to enter a customer's name, address and phone number, choose
between one and seven pizzas, select a size and toppings for each pizza, and calculate the order total. The
bill must show customer details, pizza count, each pizza and size, topping choices, individual costs, a
delivery-fee line, and the final total. Currency is displayed in two decimal places.

1.2 Data sheet supplied in the Team 1 draft

Pizza Price	Pizza size
Small	**£10.00**
Medium	**£15.00**
Large	**£20.00**

Topping	**Prize**
Sausage	**£2.00**
Chilli	**£0.50**
Ham	**£1.50**
Pineapple	**£3.30**
Olives	**£2.00**
Peperoni	**£0.50**
Tomatoes	**£0.50**

**The draft specifies seven pizza choices. This solution uses
Pizza 1 to Pizza 7 as menu labels. The base price depends on size, selected toppings are charged in addition.**

1. Analysis – requirements

Every requirement is numbered so it can be traced through design and testing.

1.3 Requirements table

ID	Requirement	Type
R1	Enter customer name, address and phone number.	**Input**
R2	Enter a whole number of pizzas from 1 to 7	**Input / rule**
R3	Select a pizza from the seven menu choices for each order item.	**Input**
R4	Select size 1-3: small, medium, or large.	**Input / rule**
R5	Select 1-7 distinct toppings for each pizza.	**Input / rule**
R6	Calculate each pizza cost and the order total from the supplied prices.	**Process**
R7	Output customer details on a personalized bill.	**Output**
R8	Output pizza count, pizza, topping names, and topping count.	**Output**
R9	Output base/topping charges, pizza totals, subtotal and final total.	**Output**
R10	Output the size of every pizza.	**Output**
R11	Output the delivery fee, assumed £0.00 pending clarification.	**Output**
R12	Display every currency value to two decimal places.	**Rule**
R13	Provide clear menus, prompts, and helpful error messages.	**Non-functional**
R14	Use efficient loops, meaningful names, and reusable functions.	**Non-functional**
R15	Reject invalid entries and ask again without terminating.	**Non-functional**

1.4 Input – Process – Output (IPO) table

Input	Process	Output
Name, address, phone (R1)	Trim outer whitespace; repeat while empty.	**Customer details (R7)**
Pizza count 1-7 (R2)	Check digits and range; repeat for many pizzas.	**Number of pizzas (R8)**
Pizza choice 1-7 (R3)	Validate; subtract 1 to obtain array index.	**Pizza label (R8**
Size choice 1-3 (R4)	Validate; look up size, name and base price.	**Size and base charge (R9-R10)**
Topping count and choices (R5)	Check count 1-7; validate IDs; reject duplicates.	**Topping names, count and charges (R8-R9)**

No price entry by staff	Base price + selected topping prices; sum items; add configured delivery fee.	**Item totals, delivery and grand total (R6, R9, R11-R12)**

1.5 Assumptions and questions for the client

Where the brief is silent, the decision taken in this solution is recorded so it can be confirmed with the client.

The brief does not say....	Decision in this solution
How many pizzas can be ordered?	**Accept between 1 and 7 pizzas per order.**
Can the same topping be selected twice?	**Allow each topping only once per pizza to prevent accidental double charging.**
Should additional VAT, taxes, or discounts apply?	**Apply none because no rules were supplied.**
How should invalid input be handled?	**Display a helpful error message and ask for the information again.**
Are online payments required?	**Calculate and display the bill without processing payments.**

1. Analysis - validation and computational thinking

1.6 Validation rules

Data item	Valued values	Response to invalid input
Name, address phone number	Text non-empty after trimming	**This cannot be blank please try again**
Pizza count/Pizza id	Whole number 1-7.	**Enter a whole number from 1 to 7.**
Size id	Whole number 1-3	**Enter a whole number from 1 to 3**
Topping count/ID	Whole number 1-7	**Enter a whole number from 1 to 7.**
Topping repetition	No repeated ID on same pizza	**The topping is already selected**

1.7 Computational thinking applied

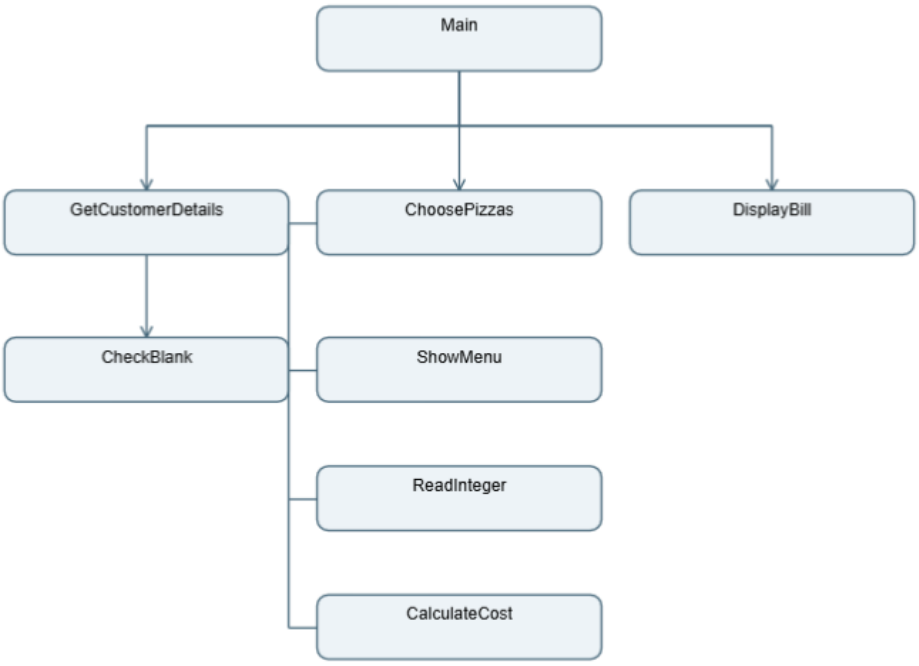
Pillar	How is was applied to this problem
Decomposition	Separate customer entry, integer validation, menu display, pizza selection, cost calculation, and bill

	output into functions.
Pattern recognition	One integer-validation loop handles all count and ID prompts. One menu function list pizza types, sizes and toppings.
Abstraction	Store only customer details and the chosen pizza/size/topping IDs and costs. Hide validation and costing behind function calls.
Algorithmic thinking	Load prices, get customer details, get pizza count, repeat selection and costing, then print the bill. Validate before storing or indexing.

2. Design - decomposition and flow

2.1 Structure chart

The diagram shows the major functions. Choose Pizzas using Show Menu, Read Integer and Calculate Cost directly; they are independent helpers. DisplayBill uses the money formatter. Main also calls ReadInteger for the order count.



2.2 Design explanation

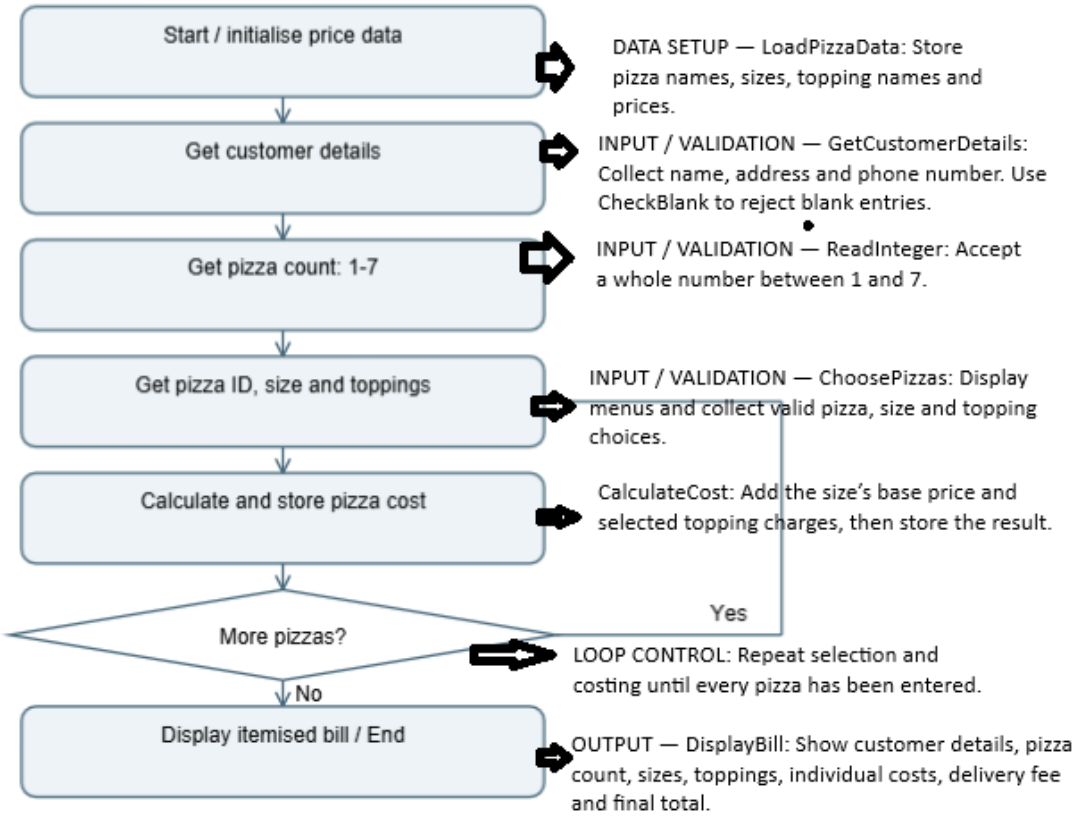
Main coordinates the order. GetCustomerDetails checks all three customer fields.

ChoosePizzas repeats exactly once per pizza and stores each completed item. DisplayBill prints the stored choices and add their cost.

Constant tuples hold the price list; this performs the LoadPizzaData role in the original structure chart.

2. Design - main flowchart

2.3 flowchart design



2.3 Data plan

Data structures (arrays) An array is one variable that holds a list of related values, each at a numbered position called an index starting at 0. A whole array can be passed to a subroutine in one step, and arrays are passed by reference, so a subroutine can fill an array that Main owns.

Arrays	Type	Size	Declared in	Holds	Example

customerDetails	String	3	Main	Customer name, address and phone number at indexes 0, 1 and 2.	`customerDetails[0] = "Sam Patel"`
pizzaNames	**String**	**7**	**Main**	**The seven pizza labels, loaded by `LoadPizzaData`.**	**`pizzaNames[0] = "Pizza 1"`**
sizePrices	**Integer**	**3**	**Main**	**Small, Medium and Large.**	**`sizeNames[1] = "Medium"`**
SizeNames	**String**	**3**	**Main**	**Base prices in pence, matching the indexes in `sizeNames`.**	**`sizePrices[1] = 1500` (£15.00)**
toppingNames	**String**	**7**	**Main**	**The seven available topping names.**	**`toppingNames[0] = "Sausage"`**
toppingPrices	**Integer**	**7**	**Main**	**Topping prices in pence, matching the indexes in `toppingNames`.**	**`toppingPrices[0] = 200` (£2.00)**
chosenPizza	**Integer**	**7**	**Main**	**The pizza-name index selected for each ordered pizza.**	**`chosenPizza[0] = 6` selects Pizza 7**
chosenSize	**Integer**	**7**	**Main**	**The size index selected for each ordered pizza.**	**`chosenSize[0] = 2` selects Large**
toppingCount	**Integer**	**7**	**Main**	**The number of toppings selected for each ordered pizza.**	**`toppingCount[0] = 2`**
chosenToppings	**Integer**	**49**	**Main**	**Selected topping indexes, with seven storage positions allocated to each pizza.**	**`chosenToppings[0] = 0` selects Sausage for the first pizza**
pizzaCost	**Integer**	**7**	**Main**	**The calculated cost of each ordered pizza, including toppings, in pence.**	**`pizzaCost[0] = 2250` (£22.50)**

Variables

Variable	Type	Used in	Purpose	Valid values / validation
Count	Integer	Main, ChoosePizzas	Stores the number of pizzas ordered.	Whole number from **1–7**.
`pizza_id/size_id`	Integer	ChoosePizzas; `size_id` also in CalculateCost	Identify the selected pizza type and size	Zero-based indexes: **0–6** for pizza; **0–2** for size.
`topping_count`	Integer	ChoosePizzas	Stores the number of toppings requested for the current pizza.	Whole number from **1–7**.

`topping_id`	Integer	ChoosePizzas	Identifies the topping selected by staff.	Index **0–6**; duplicate toppings on the same pizza are rejected.
`raw / value`	String/ Integer	ReadIntege	Store the input before validation and the converted number after validation.	Input must contain digits and be within the specified range.
`cost / subtotal`	Integer	ChoosePizzas / DisplayBill	Store an individual pizza cost and the running order total in pence.	Non-negative calculated amounts.
`DELIVERY_FEE`	Integer constant	Displaybill	Stores the delivery charge added to the subtotal.	**0 pence** explicitly assumed.
`index / number`	Integer	ShowMenu and ChoosePizzas / DisplayBill	Track menu or pizza-loop positions and number the pizzas on the bill.	Index starts at **0**; bill numbering starts at **1**.

2.4 Subroutine specifications

Subroutines	Parameters	Returns	Purpose (Requirement met)
Main	Input/output functions; defaults to keyboard input and screen output.	Integer: grand total in pence.	Coordinates customer entry, pizza selection, calculation and billing (**R1–R15**).
`GetCustomerDetails`	Input/output functions.	String list: customer name, address, and phone number.	Collects customer details and checks that none are blank (**R1, R15**).
`CheckBlank`	Prompt (String); input/output functions.	String: trimmed, non-empty text.	Rejects blank or whitespace-only entries and asks again (**R1, R13, R15**).
`ReadInteger`	Prompt (String), minimum (Integer), maximum (Integer); input/output functions.	Integer: validated number within the specified range.	Validates pizza counts, pizza choices, sizes, and topping inputs (**R2–R5, R13, R15**).
`ShowMenu`	Names (String sequence), optional	Nothing.	Displays numbered menu choices and, where supplied, formatted prices (**R3–R5, R12–R13**).

	matching prices (Integer sequence); output function.		
`ChoosePizzas`	Count (Integer); input/output functions.	List of completed pizza records.	Collects each pizza's type, size and distinct toppings, then calculates and stores its cost (**R3–R6, R15**).
`CalculateCost`	Size index (Integer), selected topping indexes (Integer list).	selected topping indexes (Integer list).Integer: individual pizza cost in pence.	Adds the size's base price and selected topping charges (**R6**).
`DisplayBill`	Customer details (String list), order records (List); output function.	Integer: final order total in pence.	Outputs customer details, pizza count, sizes, toppings, item costs, subtotal, delivery fee and final total (**R6–R12**).
`Money`	Pence (non-negative Integer).	String: currency formatted with a pound sign and two decimal places.	Formats menu and bill amounts consistently (**R12**).

2.5 Algorithm: Pseudocode

Generated in Flowgorithm and organized by function.

Function Main

```
   ... ABSTRACTION: Main coordinates customer entry, pizza selection, costing and the final
bill.
  ... DATA STRUCTURES: Declare related values in arrays.
    Declare String Array customerDetails[3]
     ... DATA STRUCTURES: Declare related values in arrays.
    Declare String Array pizzaNames[7]
     ... DATA STRUCTURES: Declare related values in arrays.
    Declare String Array sizeNames[3]
     ... DATA STRUCTURES: Declare related values in arrays.
    Declare Integer Array sizePrices[3]
    ... DATA STRUCTURES: Declare related values in arrays.
    Declare String Array toppingNames[7]
  ... DATA STRUCTURES: Declare related values in arrays.
```

```
    Declare Integer Array toppingPrices[7]
  ... DATA STRUCTURES: Declare related values in arrays.
    Declare Integer Array chosenPizza[7]
  ... DATA STRUCTURES: Declare related values in arrays.
   Declare Integer Array chosenSize[7]
    ... DATA STRUCTURES: Declare related values in arrays.
    Declare Integer Array toppingCount[7]
 ... DATA STRUCTURES: Declare related values in arrays.
    Declare Integer Array chosenToppings[49]
    ... DATA STRUCTURES: Declare related values in arrays.
    Declare Integer Array pizzaCost[7]
   ... VARIABLES: Declare the local values used in this function.
    Declare Integer noOfPizzas
    Declare Integer deliveryFee
  ... ASSUMPTION: Delivery is 0 pence until the client supplies a delivery-charge rule.
    Assign deliveryFee = 0
     ... ABSTRACTION: Call LoadPizzaData to load the supplied menu data.
    Call LoadPizzaData(pizzaNames, sizeNames, sizePrices, toppingNames, toppingPrices)
   ... ABSTRACTION: Call GetCustomerDetails to collect and validate customer details.
    Call GetCustomerDetails(customerDetails)
    ... INPUT / VALIDATION: Get a whole-number choice in the permitted range.
    Assign noOfPizzas = ReadInteger("Number of pizzas (1-7):", 1, 7)
    ... ABSTRACTION: Call ChoosePizzas to record the pizzas, sizes and toppings.
    Call ChoosePizzas(noOfPizzas, pizzaNames, sizeNames, sizePrices, toppingNames,
toppingPrices, chosenPizza, chosenSize, toppingCount, chosenToppings, pizzaCost)
    ... ABSTRACTION: Call DisplayBill to print the final itemised bill.
    Call DisplayBill(customerDetails, noOfPizzas, deliveryFee, pizzaNames, sizeNames,
sizePrices, toppingNames, toppingPrices, chosenPizza, chosenSize, toppingCount,
chosenToppings, pizzaCost)
End
```

LoadPizzaData

Function LoadPizzaData (String Array pizzaNames, String Array sizeNames, Integer Array sizePrices, String Array toppingNames, Integer Array toppingPrices)

... DATA SETUP: Store the pizza labels, size prices and topping prices in parallel arrays. All prices are in pence.

Assign pizzaNames[0] = "Pizza 1"

Assign pizzaNames[1] = "Pizza 2"

Assign pizzaNames[2] = "Pizza 3"

Assign pizzaNames[3] = "Pizza 4"

Assign pizzaNames[4] = "Pizza 5"

Assign pizzaNames[5] = "Pizza 6"

Assign pizzaNames[6] = "Pizza 7"

Assign sizeNames[0] = "Small"

```
    Assign sizeNames[1] = "Medium"
    Assign sizeNames[2] = "Large"
    Assign sizePrices[0] = 1000
    Assign sizePrices[1] = 1500
    Assign sizePrices[2] = 2000
    Assign toppingNames[0] = "Sausage"
    Assign toppingNames[1] = "Chillis"
    Assign toppingNames[2] = "Ham"
    Assign toppingNames[3] = "Pineapple"
    Assign toppingNames[4] = "Olives"
    Assign toppingNames[5] = "Pepperoni"
    Assign toppingNames[6] = "Tomatoes"
    Assign toppingPrices[0] = 200
    Assign toppingPrices[1] = 50
    Assign toppingPrices[2] = 150
    Assign toppingPrices[3] = 350
    Assign toppingPrices[4] = 200
    Assign toppingPrices[5] = 50
    Assign toppingPrices[6] = 50
End
```

GetCustomerDetails

```
Function GetCustomerDetails (String Array customerDetails)
    ... ABSTRACTION: Collect the customer name, address and phone number using one reusable
validation function.
    ... INPUT / VALIDATION: Get a non-blank customer detail.
    Assign customerDetails[0] = CheckBlank("Enter name:")

    ... INPUT / VALIDATION: Get a non-blank customer detail.
    Assign customerDetails[1] = CheckBlank("Enter address:")

    ... INPUT / VALIDATION: Get a non-blank customer detail.
    Assign customerDetails[2] = CheckBlank("Enter phone number:")
End
```

CheckBlank

```
Function CheckBlank (String prompt)
    ... VALIDATION: Keep asking until the customer details contain non-whitespace text.
    ... VARIABLES: Declare the local values used in this function.
    Declare String text
  Assign text = ""
    ... LOOP / VALIDATION: Repeat the prompt until a non-blank value is entered.
    While Len(text) = 0
        Output prompt
   ... INPUT: Read the staff member's entry as text.
        Input text
        Assign text = TrimText(text)
  ... SELECTION / VALIDATION: Check Len(text) = 0 and follow the appropriate branch.
        If Len(text) = 0
            Output "This cannot be blank. Please try again."
        End
    End
    Return String text
End
```

TrimText

```
Function TrimText (String text)
    ... VALIDATION HELPER: Remove whitespace from both ends of customer and numeric entries.
    ... VARIABLES: Declare the local values used in this function.
    Declare String result
   ... VARIABLES: Declare the local values used in this function.
    Declare Integer first
    Declare Integer last
    Declare Integer index
   ... VARIABLES: Declare the local values used in this function.
    Declare Boolean searching
    Assign first = 0
    Assign last = Len(text) - 1
    Assign searching = true
    ... LOOP / VALIDATION: Skip whitespace at the beginning or end of the text.
    While first <= last and searching

        ... SELECTION / VALIDATION: Check ToCode(Char(text, first)) <= 32 and follow the
appropriate branch.
        If ToCode(Char(text, first)) <= 32
            Assign first = first + 1
        Else
            Assign searching = false
        End
    End
    Assign searching = true

    ... LOOP: Copy the characters of the trimmed text.
    While last >= first and searching
```

```
        ... SELECTION / VALIDATION: Check ToCode(Char(text, last)) <= 32 and follow the
appropriate branch.
        If ToCode(Char(text, last)) <= 32
            Assign last = last - 1
        Else
            Assign searching = false
        End
    End
    Assign result = ""
  ... SELECTION / VALIDATION: Check first <= last and follow the appropriate branch.
    If first <= last

        ... LOOP: Repeat this section for the required values.
        For index = first to last
            Assign result = result & Char(text, index)
        End
    End
    Return String result
End
```

ReadInteger

```
Function ReadInteger (String prompt, Integer minimum, Integer maximum)
    ... VALIDATION: Check numeric text before conversion and accept only a whole number within
the specified range.
    ... VARIABLES: Declare the local values used in this function.
    Declare String raw
    ... VARIABLES: Declare the local values used in this function.
    Declare Integer value
    Declare Integer index
    Declare Integer digit
    ... VARIABLES: Declare the local values used in this function.
    Declare Boolean valid
    Declare Boolean digitsOnly
    Declare Boolean matched
    Assign valid = false
    Assign value = 0
    ... LOOP / VALIDATION: Repeat input until the entry is valid.
    While not valid
        Output prompt
        ... INPUT: Read the staff member's entry as text.
        Input raw
        Assign raw = TrimText(raw)
        Assign digitsOnly = Len(raw) >
        ... LOOP / VALIDATION: Check each character in the numeric entry.
        For index = 0 to Len(raw) - 1
            Assign matched = false
            ... LOOP / VALIDATION: Match the character against digits 0 to 9.
            For digit = 0 to 9
```

```
                ... SELECTION / VALIDATION: Check Char(raw, index) = ToString(digit) and
follow the appropriate branch.
                If Char(raw, index) = ToString(digit)
                    Assign matched = true
                End
            End

            ... SELECTION / VALIDATION: Check not matched and follow the appropriate branch.
            If not matched
                Assign digitsOnly = false
            End
        End
        Assign raw = RemoveLeadingZeros(raw)

        ... SELECTION / VALIDATION: Check digitsOnly and Len(raw) <= 1 and follow the
appropriate branch.
        If digitsOnly and Len(raw) <= 1
            Assign value = ToInteger(raw)
            Assign valid = value >= minimum and value <= maximum
        End

        ... SELECTION / VALIDATION: Check not valid and follow the appropriate branch.
        If not valid
            Output "Enter a whole number from " & ToString(minimum) & " to " &
ToString(maximum) & "."
        End
    End
    Return Integer value
End
```

RemoveLeadingZeros

```
Function RemoveLeadingZeros (String text)
    ... VALIDATION HELPER: Remove unnecessary leading zeros before numeric conversion.
    ... VARIABLES: Declare the local values used in this function.
    Declare String result

    ... VARIABLES: Declare the local values used in this function.
    Declare Integer first
    Declare Integer index

    ... VARIABLES: Declare the local values used in this function.
    Declare Boolean searching

    Assign first = 0
    Assign searching = true
```

```
    ... LOOP: Skip leading zeros while retaining at least one character.
    While first < Len(text) - 1 and searching

        ... SELECTION / VALIDATION: Check Char(text, first) = "0" and follow the appropriate
branch.
        If Char(text, first) = "0"
            Assign first = first + 1
        Else
            Assign searching = false
        End
    End
    Assign result = ""

    ... LOOP: Copy the remaining numeric text.
    For index = first to Len(text) - 1
        Assign result = result & Char(text, index)
    End
    Return String result
End
```

Show Pizza Menu

```
Function ShowPizzaMenu (String Array pizzaNames)
    ... OUTPUT: Display the seven pizza choices with menu IDs from 1 to 7.
    ... VARIABLES: Declare the local values used in this function.
    Declare Integer index

    Output "Pizza menu:"

    ... LOOP / OUTPUT: Show each pizza name and its one-based menu ID.
    For index = 0 to 6
        Output ToString(index + 1) & ". " & pizzaNames[index]
    End
End
```

Show Price Menu

```
Function ShowPricedMenu (String Array names, Integer Array prices, Integer count)
    ... OUTPUT: Display size or topping names with IDs and prices formatted to two decimal
places.
    ... VARIABLES: Declare the local values used in this function.
    Declare Integer index

    ... LOOP / OUTPUT: Show each menu entry and its formatted price.
    For index = 0 to count - 1
        Output ToString(index + 1) & ". " & names[index] & " - " & Money(prices[index])
```

```
    End
End
```

ChoosePizzas

```
Function ChoosePizzas (Integer noOfPizzas, String Array pizzaNames, String Array sizeNames,
Integer Array sizePrices, String Array toppingNames, Integer Array toppingPrices, Integer
Array chosenPizza, Integer Array chosenSize, Integer Array toppingCount, Integer Array
chosenToppings, Integer Array pizzaCost)
    ... ABSTRACTION: Record the pizza type, size and distinct toppings for every pizza in the
order.
    ... VARIABLES: Declare the local values used in this function.
    Declare Integer pizza
    Declare Integer topping
    Declare Integer candidate
    Declare Integer index


    ... VARIABLES: Declare the local values used in this function.
    Declare Boolean duplicate


    ... LOOP / DATA SETUP: Initialise all 49 topping slots.
    For index = 0 to 48


        ... DATA STORAGE: Use pizza * 7 + topping to locate a topping in the flat array.
        Assign chosenToppings[index] = -1
    End


    ... LOOP / DATA SETUP: Initialise the seven pizza record slots.
    For index = 0 to 6
        Assign chosenPizza[index] = 0
        Assign chosenSize[index] = 0
        Assign toppingCount[index] = 0
        Assign pizzaCost[index] = 0
    End


    ... LOOP: Repeat pizza selection once for every pizza ordered.
    For pizza = 0 to noOfPizzas - 1
        Output "Pizza " & ToString(pizza + 1) & " of " & ToString(noOfPizzas)


        ... ABSTRACTION: Call ShowPizzaMenu to display the pizza choices.
        Call ShowPizzaMenu(pizzaNames)


        ... INPUT / VALIDATION: Get a whole-number choice in the permitted range.
        Assign chosenPizza[pizza] = ReadInteger("Pizza choice (1-7):", 1, 7) - 1
        Output "Size menu:"


        ... ABSTRACTION: Call ShowPricedMenu to display the sizes or toppings and prices.
        Call ShowPricedMenu(sizeNames, sizePrices, 3)
```

```
        ... INPUT / VALIDATION: Get a whole-number choice in the permitted range.
        Assign chosenSize[pizza] = ReadInteger("Size (1-3):", 1, 3) - 1

        ... INPUT / VALIDATION: Get a whole-number choice in the permitted range.
        Assign toppingCount[pizza] = ReadInteger("Number of toppings (1-7):", 1, 7)
        Output "Topping menu:"

        ... ABSTRACTION: Call ShowPricedMenu to display the sizes or toppings and prices.
        Call ShowPricedMenu(toppingNames, toppingPrices, 7)
        Assign topping = 0

        ... LOOP / VALIDATION: Keep collecting distinct toppings until the requested count is
reached.
        While topping < toppingCount[pizza]

            ... INPUT / VALIDATION: Get a whole-number choice in the permitted range.
            Assign candidate = ReadInteger("Topping choice (1-7):", 1, 7) - 1
            Assign duplicate = false

            ... LOOP / VALIDATION: Compare the new topping with those already selected for
this pizza.
            For index = 0 to topping - 1

                ... SELECTION / VALIDATION: Check chosenToppings[pizza * 7 + index] =
candidate and follow the appropriate branch.
                If chosenToppings[pizza * 7 + index] = candidate
                    Assign duplicate = true
                End
            End

            ... SELECTION / VALIDATION: Check duplicate and follow the appropriate branch.
            If duplicate
                Output "That topping is already selected for this pizza. Choose another."
            Else

                ... DATA STORAGE: Use pizza * 7 + topping to locate a topping in the flat
array.
                Assign chosenToppings[pizza * 7 + topping] = candidate
                Assign topping = topping + 1
            End
        End

        ... CALCULATION: Calculate and store the complete cost of this pizza.
        Assign pizzaCost[pizza] = CalculateCost(chosenSize[pizza], toppingCount[pizza],
chosenToppings, pizza, sizePrices, toppingPrices)
    End
End
```

Calculate Cost

```
Function CalculateCost (Integer sizeId, Integer count, Integer Array chosenToppings, Integer
pizza, Integer Array sizePrices, Integer Array toppingPrices)
    ... CALCULATION: Add the selected size price and each selected topping price to obtain one
pizza cost.
    ... VARIABLES: Declare the local values used in this function.
    Declare Integer cost
    Declare Integer index

    ... CALCULATION: Set the base price or add the selected topping charge.
    Assign cost = sizePrices[sizeId]

    ... LOOP / CALCULATION: Add the price of each topping selected for this pizza.
    For index = 0 to count - 1

        ... CALCULATION: Set the base price or add the selected topping charge.
        Assign cost = cost + toppingPrices[chosenToppings[pizza * 7 + index]]
    End
    Return Integer cost
End
```

Money

```
Function Money (Integer pence)
    ... FORMATTING: Convert integer pence into pounds with a pound sign and exactly two
decimal places.
    ... VARIABLES: Declare the local values used in this function.
    Declare String formatted
    ... FORMATTING: Use ToFixed to display two decimal places.
    Assign formatted = "£" & ToFixed(pence / 100.0, 2)
    Return String formatted
End
```

DisplayBill

```
Function DisplayBill (String Array customerDetails, Integer noOfPizzas, Integer deliveryFee,
String Array pizzaNames, String Array sizeNames, Integer Array sizePrices, String Array
toppingNames, Integer Array toppingPrices, Integer Array chosenPizza, Integer Array
chosenSize, Integer Array toppingCount, Integer Array chosenToppings, Integer Array pizzaCost)
    ... OUTPUT: Produce a personalised, itemised bill showing customer details, every pizza
and the order total.
    ... VARIABLES: Declare the local values used in this function.
    Declare Integer pizza
    Declare Integer topping
    Declare Integer toppingId
    Declare Integer subtotal

    Assign subtotal = 0
    Output "PENNINE PIZZA WORKS - ITEMISED BILL"
```

```
    Output "Name: " & customerDetails[0]
    Output "Address: " & customerDetails[1]
    Output "Phone: " & customerDetails[2]

    ... OUTPUT: Show the total number of pizzas ordered.
    Output "Number of pizzas: " & ToString(noOfPizzas)

    ... LOOP / OUTPUT: Print each pizza and add its cost to the subtotal.
    For pizza = 0 to noOfPizzas - 1
        Output ToString(pizza + 1) & ". " & pizzaNames[chosenPizza[pizza]] & " - " &
sizeNames[chosenSize[pizza]]
        Output "  Base: " & Money(sizePrices[chosenSize[pizza]])
        Output "  Number of toppings: " & ToString(toppingCount[pizza])

        ... LOOP / OUTPUT: Print each selected topping and its individual charge.
        For topping = 0 to toppingCount[pizza] - 1
            Assign toppingId = chosenToppings[pizza * 7 + topping]
            Output "  " & toppingNames[toppingId] & ": " & Money(toppingPrices[toppingId])
        End
        Output "  Pizza total: " & Money(pizzaCost[pizza])

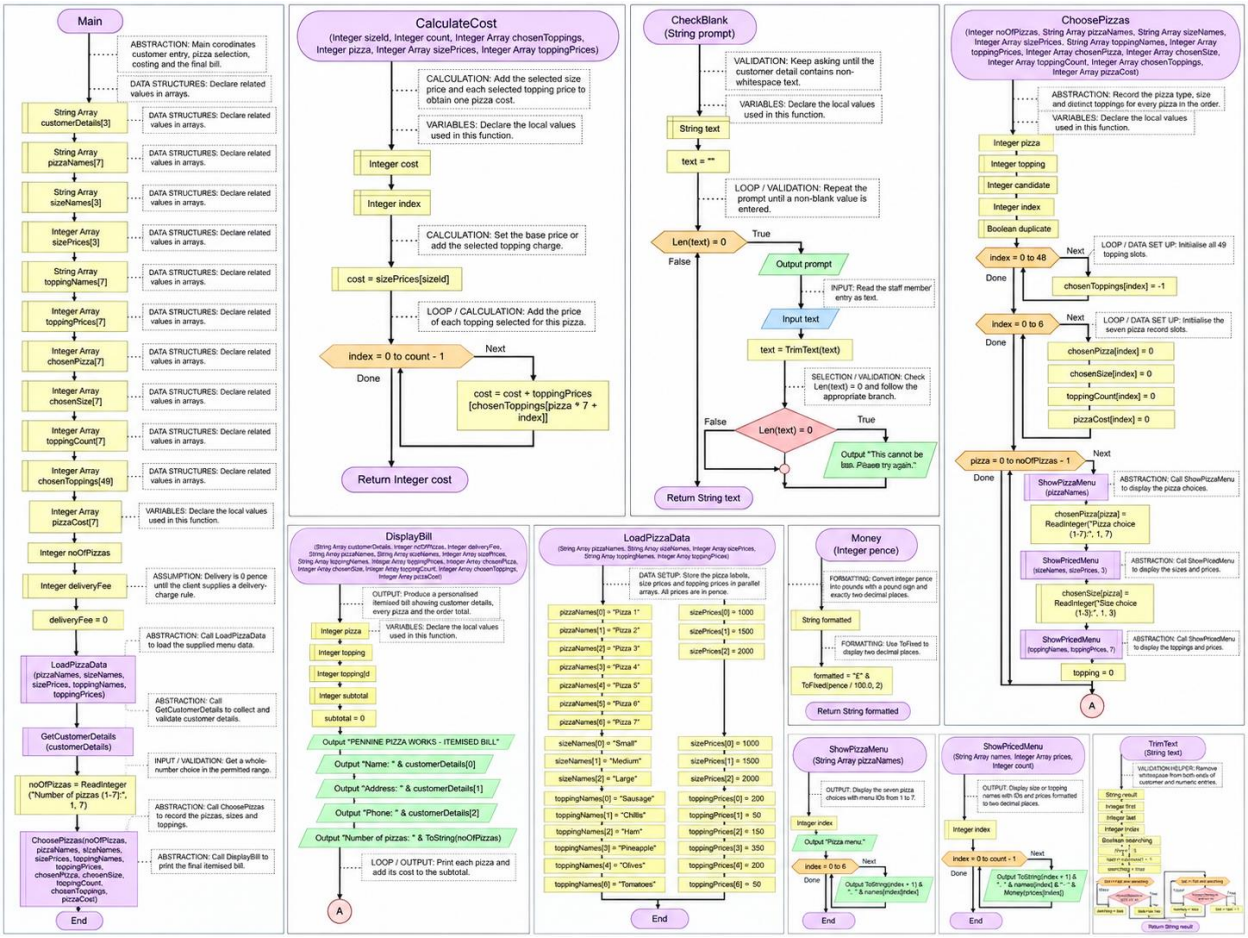
        ... CALCULATION: Add this pizza cost to the running order subtotal.
        Assign subtotal = subtotal + pizzaCost[pizza]
    End
    Output "Subtotal: " & Money(subtotal)

    ... OUTPUT: Show the delivery charge explicitly on the bill.
    Output "Delivery fee: " & Money(deliveryFee)

    ... OUTPUT / CALCULATION: Display the subtotal plus delivery as the final total.
    Output "Total: " & Money(subtotal + deliveryFee)
End
```

Flowgorithm

Sets up the data structures and calls one subroutine for each stage



3.3 How the program matches the design

Design Decision	How it was implemented
Main flowchart design	Main follows the design sequence: declare the arrays, initialise `deliveryFee`, load menu data, get customer details, get the pizza count, choose pizzas and display the bill. The pizza-selection loop shown in the design is implemented inside `ChoosePizzas`.
Subroutines	The program is divided into Flowgorithm functions, including `GetCustomerDetails`, `ReadInteger`, `ChoosePizzas`, `CalculateCost` and `DisplayBill`. Each function performs a specific task using parameters and, where needed, a return value.
Arrays	`customerDetails`, menu arrays and order arrays are declared in Main. Subroutines fill them by reference. Parallel name and price arrays use matching indexes, while

	`chosenToppings[49]` provides seven topping positions for each pizza.
Validation	`CheckBlank uses a While loop to reject blank customer details after trimming whitespace. ReadInteger checks digits and the permitted range before accepting a number. ChoosePizzas rejects duplicate toppings on the same pizza.`
Running total	`CalculateCost adds the base price and topping charges for each pizza. DisplayBill adds each stored pizza cost to subtotal inside the loop that prints the pizza, then adds the delivery fee.`
Two decimal places	`Menu prices and bill amounts use Money, which adds the pound sign and formats the amount using ToFixed(pence / 100.0, 2)`

3.4 Programming conventions followed

- **Comments** label sections by purpose, including ABSTRACTION, LOOP, VALIDATION, CALCULATION and OUTPUT.
- **Meaningful names** use camelCase, such as customerDetails, noOfPizzas and chosenToppings.
- **One main job per subroutine** makes the program easier to understand, maintain, and test.
- **Arrays** keep related information together and are passed to subroutines as parameters.
- **Variables are initialized before use.** Order arrays are initialized in ChoosePizzas, and running totals start at zero.
- **Money is stored as integer pence** to avoid rounding errors during addition.

3.5 Sample run

Input and output from a typical booking (Test 1). Lines starting with > are what the member of staff typed.

Enter name:
Oiliver Twist
Enter address:
Bolton
Enter phone number:
07773767545
Number of pizzas (1-7):
5
Pizza 1 of 5
Pizza menu:
1. Pizza 1
2. Pizza 2
3. Pizza 3
4. Pizza 4
5. Pizza 5
6. Pizza 6
7. Pizza 7
Pizza choice (1-7):
3
Size menu:
1. Small - £10.00
2. Medium - £15.00
3. Large - £20.00
Size (1-3):
2
Number of toppings (1-7):
4
Topping menu:
1. Sausage - £2.00
2. Chillis - £0.50
3. Ham - £1.50
4. Pineapple - £3.50
5. Olives - £2.00
6. Pepperoni - £0.50
7. Tomatoes - £0.50
Topping choice (1-7):
7
Topping choice (1-7):
1
Topping choice (1-7):
2
Topping choice (1-7):
3
Pizza 2 of 5
Pizza menu:
1. Pizza 1
2. Pizza 2
3. Pizza 3
4. Pizza 4
5. Pizza 5
6. Pizza 6
7. Pizza 7
Pizza choice (1-7):
6
Size menu:
1. Small - £10.00
2. Medium - £15.00
3. Large - £20.00
Size (1-3):
3
Number of toppings (1-7):

```
7
Topping menu:
1. Sausage - £2.00
2. Chillis - £0.50
3. Ham - £1.50
4. Pineapple - £3.50
5. Olives - £2.00
6. Pepperoni - £0.50
7. Tomatoes - £0.50
Topping choice (1-7):
5
Topping choice (1-7):
4
Topping choice (1-7):
1
Topping choice (1-7):
3
Topping choice (1-7):
7
Topping choice (1-7):
6
Topping choice (1-7):
2
Pizza 3 of 5
Pizza menu:
1. Pizza 1
2. Pizza 2
3. Pizza 3
4. Pizza 4
5. Pizza 5
6. Pizza 6
7. Pizza 7
Pizza choice (1-7):
6
Size menu:
1. Small - £10.00
2. Medium - £15.00
3. Large - £20.00
Size (1-3):
1
Number of toppings (1-7):
1
Topping menu:
1. Sausage - £2.00
2. Chillis - £0.50
3. Ham - £1.50
4. Pineapple - £3.50
5. Olives - £2.00
6. Pepperoni - £0.50
7. Tomatoes - £0.50
Topping choice (1-7):
2
Pizza 4 of 5
Pizza menu:
1. Pizza 1
2. Pizza 2
3. Pizza 3
4. Pizza 4
5. Pizza 5
6. Pizza 6
7. Pizza 7
```

```
Pizza choice (1-7):
3
Size menu:
1. Small - £10.00
2. Medium - £15.00
3. Large - £20.00
Size (1-3):
3
Number of toppings (1-7):
7
Topping menu:
1. Sausage - £2.00
2. Chillis - £0.50
3. Ham - £1.50
4. Pineapple - £3.50
5. Olives - £2.00
6. Pepperoni - £0.50
7. Tomatoes - £0.50
Topping choice (1-7):
4
Topping choice (1-7):
3
Topping choice (1-7):
2
Topping choice (1-7):
6
Topping choice (1-7):
4
That topping is already selected for this pizza. Choose another.
Topping choice (1-7):
1
Topping choice (1-7):
3
That topping is already selected for this pizza. Choose another.
Topping choice (1-7):
4
That topping is already selected for this pizza. Choose another.
Topping choice (1-7):
5
Topping choice (1-7):
6
That topping is already selected for this pizza. Choose another.
Topping choice (1-7):
7
Pizza 5 of 5
Pizza menu:
1. Pizza 1
2. Pizza 2
3. Pizza 3
4. Pizza 4
5. Pizza 5
6. Pizza 6
7. Pizza 7
Pizza choice (1-7):
7
Size menu:
1. Small - £10.00
2. Medium - £15.00
3. Large - £20.00
Size (1-3):
3
```

```
Number of toppings (1-7):
2
Topping menu:
1. Sausage - £2.00
2. Chillis - £0.50
3. Ham - £1.50
4. Pineapple - £3.50
5. Olives - £2.00
6. Pepperoni - £0.50
7. Tomatoes - £0.50
Topping choice (1-7):
3
Topping choice (1-7):
4
PENNINE PIZZA WORKS - ITEMISED BILL
Name: Oiliver Twist
Address: Bolton
Phone: 07773767545
Number of pizzas: 5
1. Pizza 3 - Medium
  Base: £15.00
  Number of toppings: 4
  Tomatoes: £0.50
  Sausage: £2.00
  Chillis: £0.50
  Ham: £1.50
  Pizza total: £19.50
2. Pizza 6 - Large
  Base: £20.00
  Number of toppings: 7
  Olives: £2.00
  Pineapple: £3.50
  Sausage: £2.00
  Ham: £1.50
  Tomatoes: £0.50
  Pepperoni: £0.50
  Chillis: £0.50
  Pizza total: £30.50
3. Pizza 6 - Small
  Base: £10.00
  Number of toppings: 1
  Chillis: £0.50
  Pizza total: £10.50
4. Pizza 3 - Large
  Base: £20.00
  Number of toppings: 7
  Pineapple: £3.50
  Ham: £1.50
  Chillis: £0.50
  Pepperoni: £0.50
  Sausage: £2.00
  Olives: £2.00
  Tomatoes: £0.50
  Pizza total: £30.50
5. Pizza 7 - Large
  Base: £20.00
  Number of toppings: 2
  Ham: £1.50
  Pineapple: £3.50
  Pizza total: £25.00
Subtotal: £116.00
```

```
Delivery fee: £0.00
Total: £116.00
```

4.1 Test plan and results

#	Type	Purpose and test data	Expected result	Actual result	Result
1	Normal	Order five pizzas with different sizes and toppings.	Five pizzas listed on an itemised bill; total £116.00.	All five pizzas listed; total £116.00.	**PASS**
2	Boundary	Select seven distinct toppings for order items 2 and 4.	Accept seven toppings; each large pizza costs £30.50.	Both pizzas have seven toppings and cost £30.50 each.	**PASS**
3	Boundary	Select one topping for order item 3.	Accept one topping; small pizza costs £10.50.	One topping accepted; pizza total £10.50.	**PASS**
4	Abnormal	Select toppings already chosen for order item 4.	Reject duplicates, ask again and avoid charging twice.	Four duplicate attempts rejected; only seven distinct toppings billed.	**PASS**
5	Normal	Order Pizza 6 twice, with different sizes and toppings.	Accept both as separate order items.	separate order items.Pizza 6 listed separately as Large and Small.	**PASS**
6	Normal	Check customer details and pizza count on the bill.	count on the bill.Display Oiliver Twist, Bolton, 07773767545 and a pizza count of 5.	All entered details and the count displayed correctly.	**PASS**
7	Normal	Check currency formatting on menus and bill.	Display all amounts with a pound sign and two decimal places.	Amounts shown correctly, including £116.00 total and £0.00 delivery.	**PASS**